

# Viva preparation notes

Verified Docker Desktop run | 2026-09-23 | UTC and Pakistan Standard Time (UTC+05:00)

## Viva notes

**What problem does this project solve?** Students reserve campus study rooms. The key invariant is that confirmed reservations for one room cannot overlap. The app is small so we can focus on observing reliability and load behavior.

**Why Counter versus Gauge?** A Counter accumulates events such as accepted bookings and resets only when its process restarts. Use `rate` for events per second. A Gauge describes a current value that can rise or fall, such as in-flight requests or active reservations. Never decrement a Counter to cancel a booking; increment a separate cancellation Counter.

**Why Histogram versus Summary?** Both track counts and summed observations. A Histogram also counts observations in cumulative buckets, which Prometheus can aggregate and use to estimate quantiles. This Python Summary exposes only `sum/count`; it does not compute quantiles. We use it for mean DB operation time, and a Histogram for HTTP tails.

**How does p95 work?** p95 is the latency below which approximately 95% of observations fall. Roughly 5% are slower. p99 focuses on the slowest 1% boundary. `histogram_quantile` estimates it from bucket rates, not by reading a single request's duration. Preserve `1e` when aggregating buckets. Wider buckets and sparse samples reduce precision.

**Why do p95/p99 matter?** An average may look fast while a meaningful minority of users waits a long time. Our fault affects every fifth request; the mean alone hides its distribution. Tail percentiles reveal affected users, but unstable p99 from very few samples should not be overinterpreted.

**What does `[5m]` mean?** It selects the last five minutes of samples at each evaluation time. `rate(counter[5m])` estimates per-second growth across that window and handles resets. It is neither a refresh period nor the total event count. `increase` estimates the total increase over the window. We scrape every five seconds.

**What is cardinality?** The number of distinct metric label combinations (time series). Three methods  $\times$  five routes  $\times$  four statuses can already create 60 series for one simple metric. Histogram bucket labels multiply this further. A request ID introduces new combinations continuously rather than reusing a bounded vocabulary.

**Why is request ID a terrible metric label but a useful log field?** Metrics should aggregate repeated events into a small set of series. Every new ID creates another series with storage and memory overhead. Logs represent individual events, so a searchable request ID connects an error/fault to its completion record. Our isolated experiment is capped at 100 and never changes production metrics.

**What happens after removing the label?** After the exporter restarts and Prometheus scrapes again, current counter-value series fall from 100 to 1. Historical series remain queryable in the earlier time range until retention removes them. The companion `_created` metric makes total series overhead greater than the counter-value count alone.

**Push versus pull?** The API exposes metrics; Prometheus pulls them periodically. If Prometheus is down the app does not wait for it. Filebeat pushes decoded logs to Elasticsearch. k6 pushes test metrics to Prometheus's explicitly enabled remote-write receiver. Different signals use different transport models.

**How does Prometheus obtain and store data?** It GETs `/metrics`, parses the text, attaches target labels, and appends timestamped samples to a local WAL and TSDB. Named storage survives container recreation. Failure creates collection gaps; no tool can reconstruct all unobserved distributions later.

**What does Grafana store?** Dashboard definitions, users, settings and datasource configuration. Its datasource queries Prometheus for metrics. It is not our metric database. Provisioned JSON comes from the repository so dashboards can be recreated automatically.

**Filebeat / Elasticsearch / Kibana roles?** Filebeat follows JSONL files and remembers offsets, decodes fields, and ships bulk batches. Elasticsearch indexes and stores searchable documents and applies ILM. Kibana queries Elasticsearch and supplies Discover and saved views. Kibana is not the log shipper and Filebeat is not durable infinite storage.

**Why shared log volume?** Docker Desktop's internal container-log filesystem paths are not portable Windows paths. App-owned JSONL on a named volume gives Filebeat a predictable Linux path and survives container recreation. App stdout remains useful for `docker compose logs`. File rotation bounds local storage.

**What happens if an observability component dies?** Grafana or Kibana: display unavailable; ingestion can continue. Prometheus: metrics collection gaps, booking unaffected. Filebeat: local files continue, search delivery delayed. Elasticsearch: Filebeat backs off with finite buffers; long outages can lose rotated logs. Node Exporter: resource graphs absent, booking unaffected. PostgreSQL is different: booking operations depend on it and fail cleanly while liveness remains available. See report's failure matrix.

**Can observability still hurt the app?** Yes: CPU, disk, memory, or a full machine can affect everything. The design removes synchronous remote telemetry calls from API work and bounds queues/files/containers. It does not claim absolute resource isolation. Dropped/error counters expose some telemetry failure, but if the app itself cannot be scraped those counters are unavailable too.

**Latency versus throughput?** Latency is time for an operation. Throughput is completed operations per second. A server can have high throughput and poor latency because many requests run at once. A slow test generator can also limit measured throughput, so resource measurements matter.

**Throughput versus concurrency?** Concurrency is how much work is in flight now. Throughput is completion rate. In a stable system, Little's Law relates them:  $\text{average in-flight} \approx \text{throughput} \times \text{average time in system}$ . More VUs do not guarantee more throughput once a bottleneck saturates. Our closed-loop users wait for results before sending more.

**Why connection pools?** Opening a DB connection is expensive, so a pool reuses a bounded number. When busy, requests wait briefly for a connection instead of creating unlimited ones. The app uses one async engine, pool 10 plus 5 overflow, and a 2-second checkout timeout.

**Why not a huge pool?** It can overload DB CPU/memory, increase lock contention, and worsen latency. A queue in the application may be healthier than uncontrolled parallel SQL. Tune only after observing throughput, DB time, errors, and resource usage. Our Summary includes pool wait, so it cannot alone prove that SQL execution was slow.

**What is the booking race?** Two requests can both read "available" before either inserts. An application pre-check is insufficient. The database exclusion constraint rejects intersecting UTC ranges for the same room among confirmed rows. A failed transaction becomes HTTP 409. Adjacent `[start,end)` intervals do not intersect. Direct SQL is also protected.

**How do we prove concurrency safety?** A real PostgreSQL integration test sends 20 simultaneous requests for the same slot, asserts exactly one 201 and nineteen 409s, then counts one confirmed database row. Another test bypasses the service and asserts PostgreSQL rejects direct overlap. A mock cannot prove this database property.

**How is cancellation safe under concurrency?** The SQL update only matches confirmed rows and returns whether it changed one. Multiple callers may all receive the already-cancelled representation, but only one committed transition increments the cancellation Counter. The cancelled row no longer participates in the partial exclusion constraint.

**What is our bottleneck likely to be?** A hypothesis, not a measured conclusion: the single Python worker, the bounded DB pool, DB/index/commit work, log volume, or VM resource pressure from ELK. Compare client/HTTP tails to DB means, app and VM CPU/RSS, throughput and errors before naming a cause. The assignment report has placeholders wherever measurements are absent.

**Why does the injected delay not necessarily increase CPU?** `asyncio.sleep` yields the event loop. Requests remain in flight while consuming little CPU. We should see higher HTTP duration and fault counters/logs without necessarily higher SQL latency. Fixed VUs may generate fewer requests because their iterations take longer.

**Why reconstruct active bookings from DB?** An in-memory create-minus-cancel count would be wrong after process restart or when bookings expire. A five-second DB count observes persisted state. A freshness/success metric warns when it is stale. It counts future confirmed reservations, not currently occupied rooms.

**What would change in production?** Authentication and booking ownership, TLS, least-privilege DB user, real secret management, safe upgrade migrations, backups and tested restores, durable idempotency keys, security-patched pinned images/digests, HA if justified, capacity/SLO evidence, alerts, stronger log-delivery guarantees, and explicit per-replica metrics. Adding workers requires changing logging/metric aggregation, not just increasing a command-line number.

**What are we honest about?** Docker Desktop Node Exporter observes Linux, not Windows. Histograms approximate percentiles. Missing data is not zero. Container configuration is not runtime proof. ILM attachment is not seven days of observed deletion. Benchmarks apply only to the documented hardware, resource allocation, duration, and workload. Screenshots must come from actual runs.